

Exercise 5.4

The method removes items from the collection, so a for-each loop cannot be used. A while loop is the most general type of loop and it can be used in place of any of the other types.

Exercise 5.7

```
sightings.filter(name == elephant)
          .filter(spotter == spotterID)
          .filter(period == dayID)
          .reduce(count elements);
```

Exercise 5.8

```
sightings.filter(count > 0)
```

Exercise 5.9

One possible ordering is:

```
tracks.filter(artist == Big Bill Broonzy)
       .map(played count)
       .reduce(sum)
```

Exercise 5.11

```
/**
 * Print details of the sightings on the given day.
 * @param dayID The period of interest.
 */
public void printSightingsOnDay(int dayID)
{
    sightings.stream()
              .filter(record -> record.getPeriod() == dayID)
              .forEach(record ->
System.out.println(record.getDetails()));
}
```

Exercise 5.12

```
/**
 * Print details of the sightings on the given day.
 * @param animal The animal of interest.
 * @param dayID The period of interest.
 */
public void printSightingsOfAnimalOnDay(String animal, int
dayID)
{
    sightings.stream()
              .filter(record ->
animal.equals(record.getAnimal()))
              .filter(record -> record.getPeriod() == dayID)
```

```

        .forEach(record ->
System.out.println(record.getDetails()));
    }

```

Exercise 5.13

The order does not matter since all details printed must have the property that both the animal and the period match the parameter values. Each filter operation removes all those records from the stream that do not match one of the parameters.

Exercise 5.15

```

/**
 * Print the counts of the given animal.
 * @param animal The animal of interest.
 */
public void printCountsOfAnimal(String animal)
{
    sightings.stream()
        .filter(record ->
animal.equals(record.getAnimal()))
        .forEach(record ->
System.out.println(record.getCount()));
}

```

Exercise 5.16

The order does matter in many cases. Since the output stream of a map operation typically is of a different type from its input stream, the appropriate position of the filter operation will depend on the type of its input stream.

Exercise 5.17

```

/**
 * Print a list of the types of animal considered to be
endangered.
 * @param animalNames A list of animals names.
 * @param dangerThreshold Counts less-than or equal-to to this
level
 *
        are considered to be dangerous.
 */
public void printEndangered(ArrayList<String> animalNames,
        int dangerThreshold)
{
    animalNames.stream()
        .filter(animal -> getCount(animal) <=
dangerThreshold)
        .forEach(animal -> System.out.println(animal +
" is endangered."));
}

```

Exercise 5.18

Use of the :: syntax in the example is equivalent to the original lambda expression. See Chapter 6 for the various ways in which it can be used.

Exercise 5.20

```
/**
 * Return a count of the number of sightings of the given
animal
 * by a particular spotter on a particular day.
 * @param animal The type of animal.
 * @param spotterID The spotter.
 * @param dayID When.
 * @return The count of sightings of the given animal.
 */
public int getCount(String animal, int spotterID, int dayID)
{
    return sightings.stream()
        .filter(sighting ->
animal.equals(sighting.getAnimal()))
        .filter(sighting -> spotterID ==
sighting.getSpotter())
        .filter(sighting -> dayID ==
sighting.getPeriod())
        .map(sighting -> sighting.getCount())
        .reduce(0, (runningSum, count) ->
runningSum + count);
}
```

Exercise 5.21

```
/**
 * Return a String giving the names of the animals seen by
 * a particular spotter on a particular day.
 * @param spotterID The spotter.
 * @param dayID When.
 * @return The animal names.
 */
public String seenBySpotterOnDay(int spotterID, int dayID)
{
    return sightings.stream()
        .filter(sighting -> sighting.getSpotter()
== spotterID)
        .filter(sighting -> sighting.getPeriod()
== dayID)
        .filter(sighting -> sighting.getCount() >
0)
        .map(sighting -> sighting.getAnimal())
        .reduce("", (list, animal) -> list + " " +
animal).trim();
}
```

Exercise 5.22

```
/**
 * Remove from the sightings list all of those records with
 * a count of zero.
```

```

    */
    public void removeZeroCounts()
    {
        sightings.removeIf(record -> record.getCount() == 0) ;
    }

```

Exercise 5.23

```

/**
 * Remove from the sightings list all of those records
 * from the given spotter.
 * @param spotter The ID of the spotter.
 */
public void removeBySpotter(int spotter)
{
    sightings.removeIf(record -> record.getSpotter() ==
spotter) ;
}

```

Exercise 5.25

```

/**
 * Count the number of sighting records by the given spotter.
 * @param spotter The ID of the spotter.
 * @return The number of sighting records.
 */
public long countBySpotter(int spotter)
{
    return sightings.stream()
        .filter(record -> record.getSpotter() ==
spotter)
        .count();
}

```

Exercise 5.26

This is slightly tricky because max returns an OptionalInt. In other words, there might be no matching animal in the stream, so a default return value is required in that case.

```

/**
 * Return the largest count in a sighting of the given animal.
 * @param animal The animal of interest.
 * @return The largest count.
 */
public int getLargestCount(String animal)
{
    return sightings.stream()
        .filter(record ->
animal.equals(record.getAnimal()))
        .map(record -> record.getCount())
        .max((x, y) -> x - y)
        .orElse(0);
}

```

Exercise 5.27

```

/**

```

```

    * Find the first sighting record for the given
    * spotter and animal.
    * @param spotter The spotter.
    * @param animal The animal of interest.
    * @return The sighting, or null if there are none.
    */
    public Sighting findFirstSightingBy(int spotter, String
animal)
    {
        return sightings.stream()
                        .filter(record -> record.getSpotter() ==
spotter)
                        .filter(record ->
animal.equals(record.getAnimal()))
                        .findFirst()
                        .orElse(null);
    }

```

Exercise 5.29

```

/**
 * Show a list of all the tracks in the collection.
 */
public void listAllTracks()
{
    System.out.println("Track listing: ");
    tracks.forEach(track ->
System.out.println(track.getDetails()));
    System.out.println();
}

/**
 * List all tracks by the given artist.
 * @param artist The artist's name.
 */
public void listByArtist(String artist)
{
    tracks.stream()
        .filter(track -> track.getArtist().contains(artist))
        .forEach(track ->
System.out.println(track.getDetails()));
}

```